NAME: SUBASH CHANDRA BOSE

Cybersecurity Attack Lab Report: DNS Spoofing, ARP Poisoning, DoS and Brute Force Analysis

Executive Summary

This report documents a comprehensive network security lab demonstrating four major attack vectors: DNS spoofing, ARP poisoning, and brute force authentication and DoS attacks. The lab environment consists of Kali Linux (attacker) operating as a Man-in-the-Middle (MITM) against an Ubuntu victim machine, with all network traffic and attacks captured and analyzed through Splunk logs.

Lab Duration: December 24, 2025
Attacker Machine: Kali Linux (192.168.202.5)
Victim Machine: Ubuntu 15 (192.168.202.3)
Gateway/Router: 192.168.202.2
Network: Host-Only Adapter (192.168.202.0/24)
SIEM: Splunk Enterprise

---

1. Lab Environment Setup

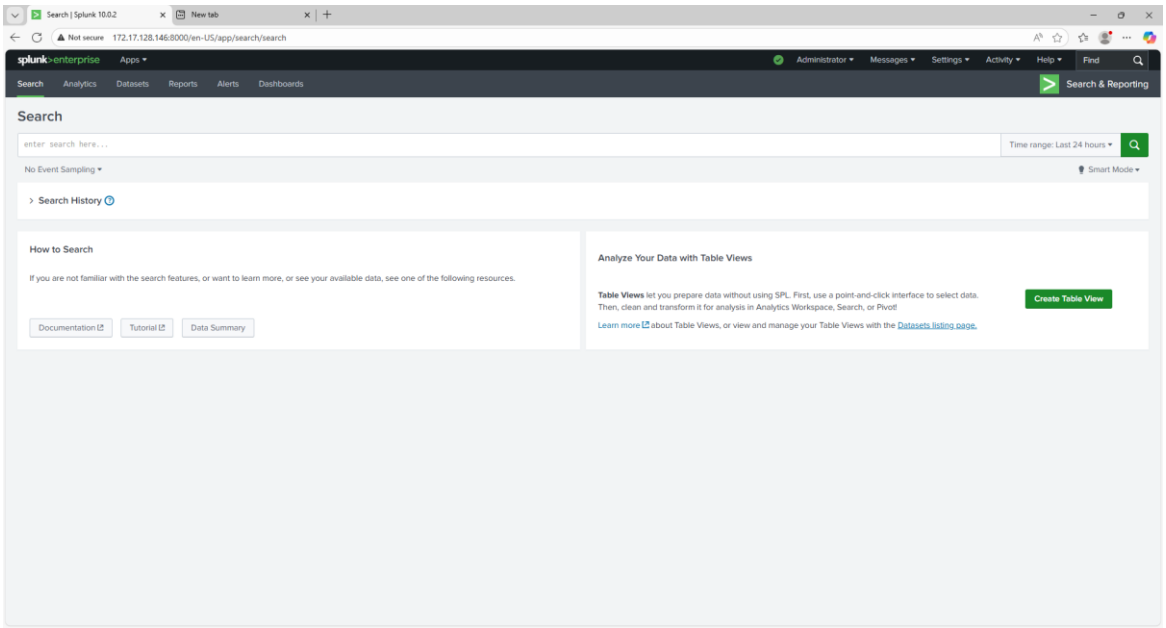
1.1 Network Configuration

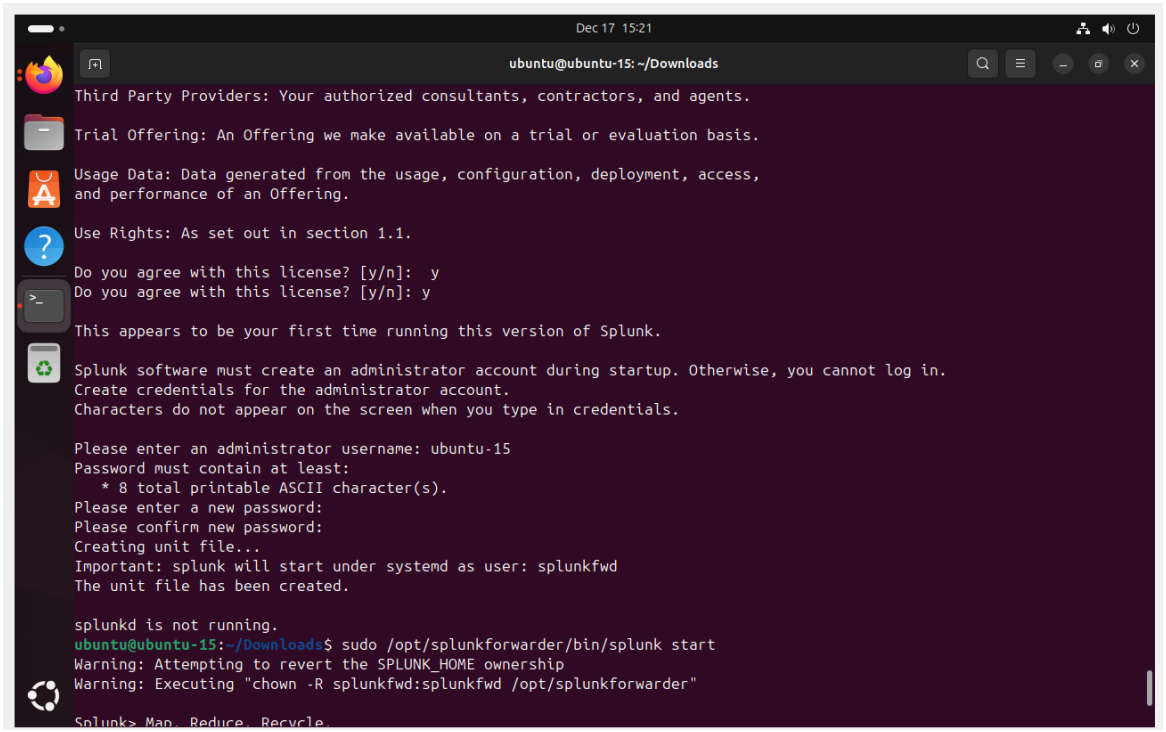
The lab uses a virtualized network environment with the following specifications:

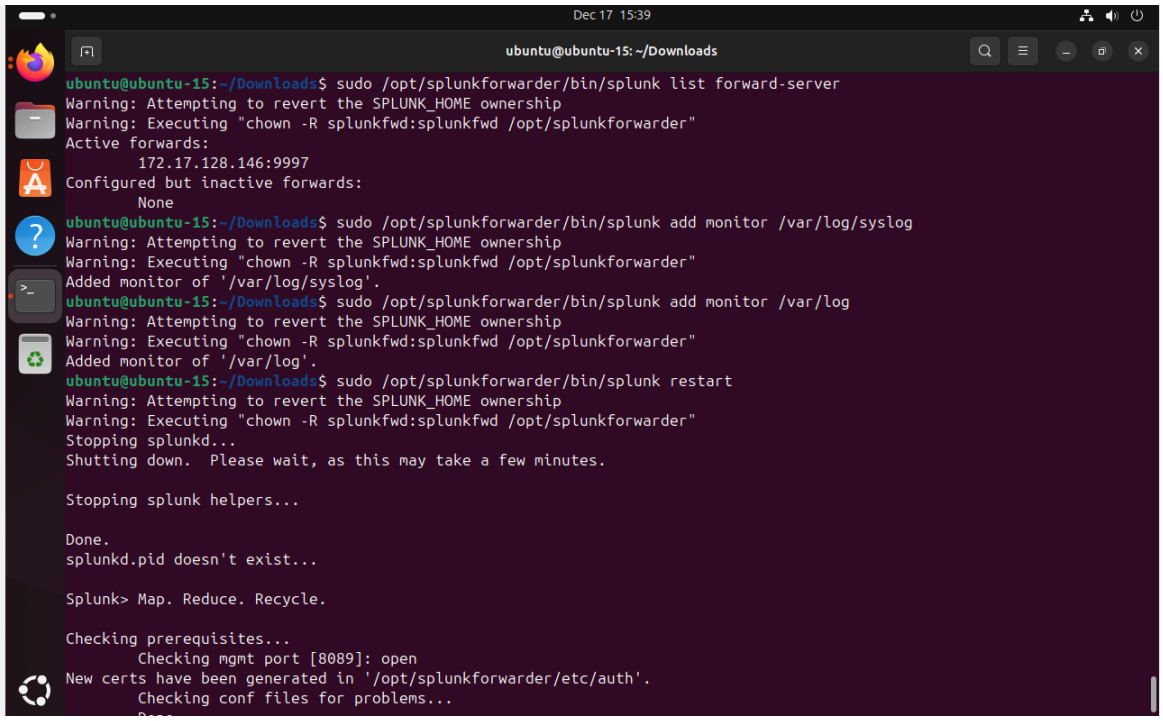
Component	Details
Network Type	Host-Only Adapter
Network Range	192.168.202.0/24
Gateway IP	192.168.202.2
Kali Machine	192.168.202.5/24 (Attacker)
Ubuntu Machine	192.168.202.3/24 (Victim)

Primary DNS	192.168.202.2 (Host-Only Gateway)

CONFIGURATION OF SPLUNK PORT FORWARDER:







1.2 Prerequisite Configuration

Before executing the attacks, the victim machine (Ubuntu) was configured to route all DNS requests through the host-only network interface (`enp0s3`) instead of the NAT adapter. This critical step ensures DNS queries are sent over the vulnerable network where the attacker can intercept them.

Ubuntu DNS Configuration:

- IPv4 Address: 192.168.202.3/24
- DNS Server: 192.168.202.2 (host-only gateway)
- Automatic DNS: Disabled
- Connection: Set to Connect Automatically

Verification Command:
nmcli dev show | grep DNS

Output: Shows DNS[1]: 192.168.202.2 only (no NAT DNS leakage)

This configuration is critical because it prevents Ubuntu from using the NAT adapter's DNS (which would bypass the attack lab entirely).

---

2. Attack 1: ARP Spoofing and Man-in-the-Middle Setup

2.1 Attack Overview

Address Resolution Protocol (ARP) spoofing is a network layer attack that allows an attacker to intercept traffic by poisoning the ARP cache of both the victim and gateway. By claiming to be the gateway to the victim and the victim to the gateway, the attacker becomes the MITM.

Attack Flow:
Ubuntu (192.168.202.3) ← ARP poison → believes 192.168.202.5 is gateway 192.168.202.2
Gateway (192.168.202.2) ← ARP poison → believes 192.168.202.5 is Ubuntu 192.168.202.3
Result: All traffic between Ubuntu and Gateway flows through Kali (192.168.202.5)

2.2 Attack Execution: Ettercap

Tool Used: Ettercap (GUI mode)
Purpose: ARP spoofing and MITM interception

Step 1: Start Ettercap
sudo ettercap -G

Step 2: Configure MITM Attack

- Navigate: `Sniff → Unified Sniffing` → Select `eth0` interface
- Go to: `MITM → ARP Spoofing`
- Target 1: Gateway (192.168.202.2)
- Target 2: Ubuntu victim (192.168.202.3)
- Start the attack

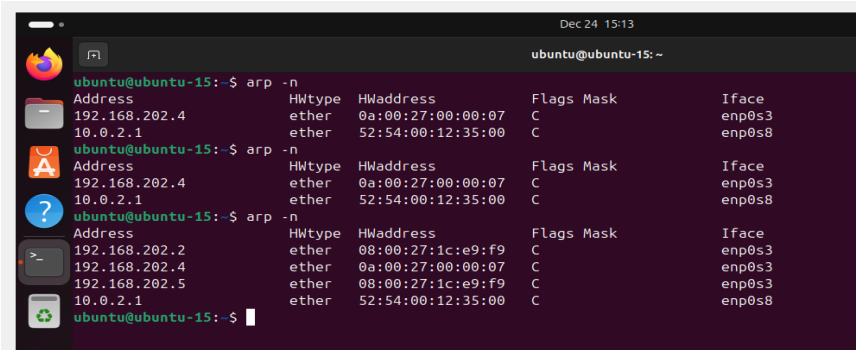
Step 3: Verification
The Ettercap main window displays:

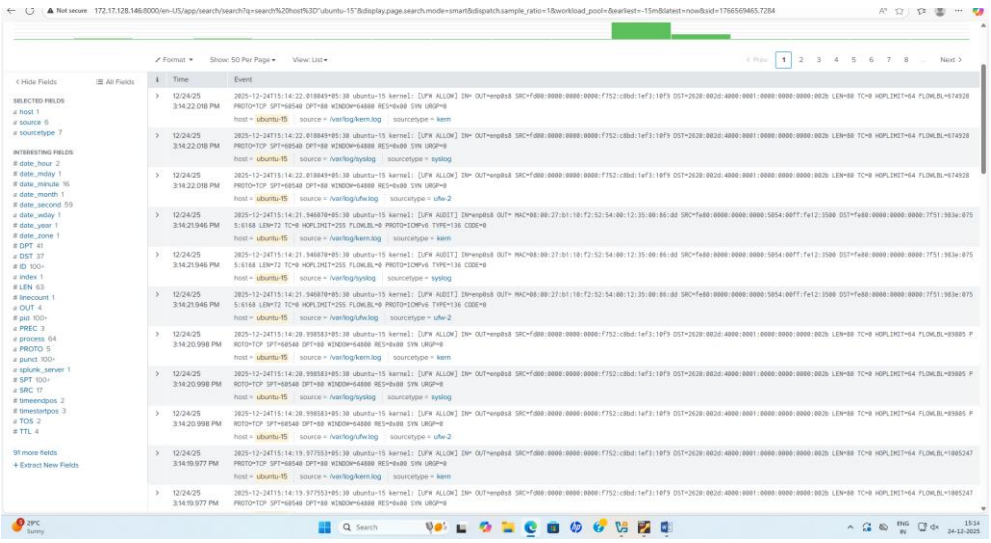
- **Active connections:** Shows bidirectional traffic between Ubuntu and Gateway
- **ARP poisoning status:** "Poison in progress" indicator
- **Captured traffic:** HTTP, DNS, and SSH packets flowing through Kali

Evidence Screenshot:
Shows Ettercap running with active MITM connections, poisoning Ubuntu and gateway simultaneously.

```
File  Actions  Edit  View  Help
┌──(kali㉿kali)-[~]
└─$ sudo ettercap -T -q -M arp:remote /192.168.202.3// /192.168.202.2//
[sudo] password for kali:

ettercap 0.8.3.1 copyright 2001-2020 Ettercap Development Team

Listening on:
  eth0 -> 08:00:27:1C:E9:F9
          192.168.202.5/255.255.255.0
          fe80::a00:27ff:fe1c:e9f9/64

SSL dissection needs a valid 'redir_command_on' script in the etter.conf file
Ettercap might not work correctly. /proc/sys/net/ipv6/conf/eth0/use_tempaddr
is not set to 0.
Privileges dropped to EUID 65534 EGID 65534...

  34 plugins
  42 protocol dissectors
  57 ports monitored
28230 mac vendor fingerprint
1766 tcp OS fingerprint
2182 known services
Lua: no scripts were specified, not starting up!

Scanning for merged targets (2 hosts)...

* |==================================================>| 100.00 %

2 hosts added to the hosts list...

ARP poisoning victims:

 GROUP 1 : 192.168.202.3 08:00:27:8B:CC:3A

 GROUP 2 : 192.168.202.2 08:00:27:0B:D5:0E
Starting Unified sniffing...


Text only Interface activated...
Hit 'h' for inline help

DHCP: [08:00:27:8B:CC:3A] REQUEST 192.168.202.3
DHCP: [192.168.202.2] ACK : 192.168.202.3 255.255.255.0 GW invalid
DHCP: [08:00:27:8B:CC:3A] REQUEST 192.168.202.3
DHCP: [192.168.202.2] ACK : 192.168.202.3 255.255.255.0 GW invalid
```

```
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup default qlen 1000
    link/ether 08:00:27:1c:e9:f9 brd ff:ff:ff:ff:ff:ff
    inet 192.168.202.5/24 brd 192.168.202.255 scope global dynamic noprefixro
ute eth0
       valid_lft 399sec preferred_lft 399sec
    inet6 fe80::a00:27ff:fe1c:e9f9/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```





---

3. Attack 2: DNS Spoofing

3.1 DNS Spoofing Overview

DNS spoofing (DNS hijacking) is a Layer 7 attack that intercepts DNS queries and returns malicious responses. Combined with ARP spoofing, it allows the attacker to redirect users to fake websites.

Attack Chain:

1. Ubuntu user enters `mybank.lab` in browser
2. Ubuntu sends DNS query "What is the IP of mybank.lab?"
3. Query travels through Kali (due to ARP spoofing)
4. dnsspoof intercepts the query
5. dnsspoof returns malicious response: `mybank.lab = 192.168.202.5` (Kali's IP)

6. Ubuntu browser connects to Kali's Apache server (believing it is mybank.lab)
7. User sees fake banking website hosted on Kali

3.2 DNS Spoofing Setup

Tools Used:

- **Ettercap:** For ARP spoofing (prerequisite)
- **dnsspoof:** For DNS query interception and malicious responses
- **Apache2:** For hosting fake website on Kali

Step 1: Create Hosts File for dnsspoof

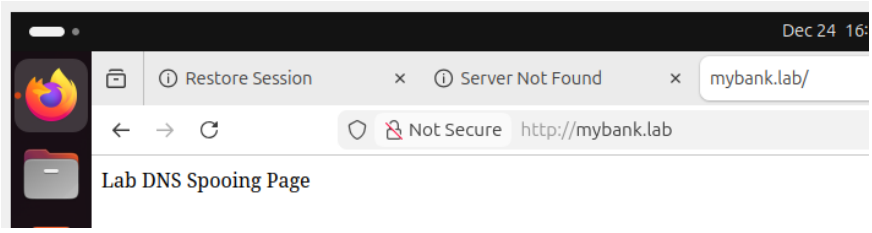
On Kali, create `/etc/dnsspoof/hosts` file:
192.168.202.5 mybank.lab

This tells dnsspoof to resolve these domains to Kali's IP.

Step 2: Create Fake Website

On Kali, create Apache web root with fake banking page:
sudo mkdir -p /var/www/html/mybank

Create `/var/www/html/index.html`:



Enable Apache:
sudo systemctl start apache2
sudo systemctl enable apache2

Step 3: Start dnsspoof

On Kali, with Ettercap ARP spoofing active:
sudo dnsspoof -i eth0 -f /etc/dnsspoof/hosts

Parameters:

- `-i eth0`: Listen on eth0 interface
- `-f /etc/dnsspoof/hosts`: Use custom hosts file for spoofing

Output: dnsspoof displays intercepted DNS queries and spoofed responses:
listening on eth0

192.168.202.3 > 192.168.202.2: mybank.lab (A) ?
192.168.202.2 > 192.168.202.3: mybank.lab (A) 192.168.202.5 [spoofed]

3.3 DNS Spoofing Verification

On Ubuntu victim:

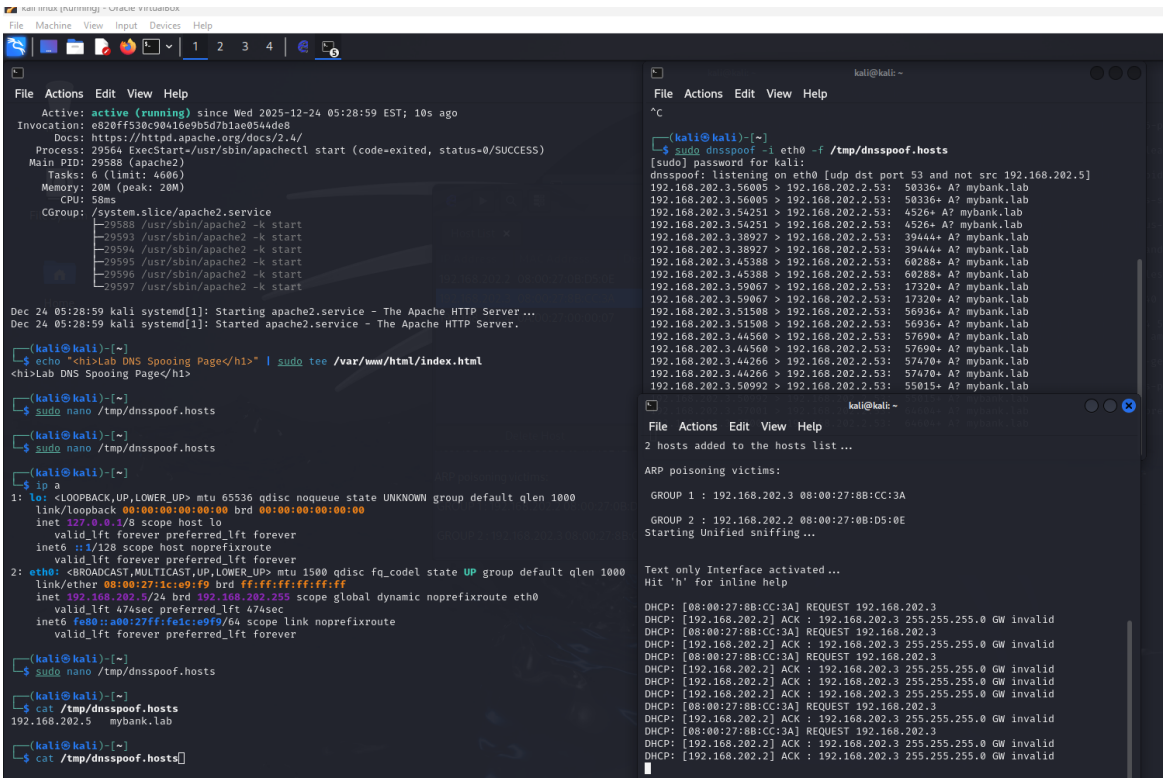
dig mybank.lab

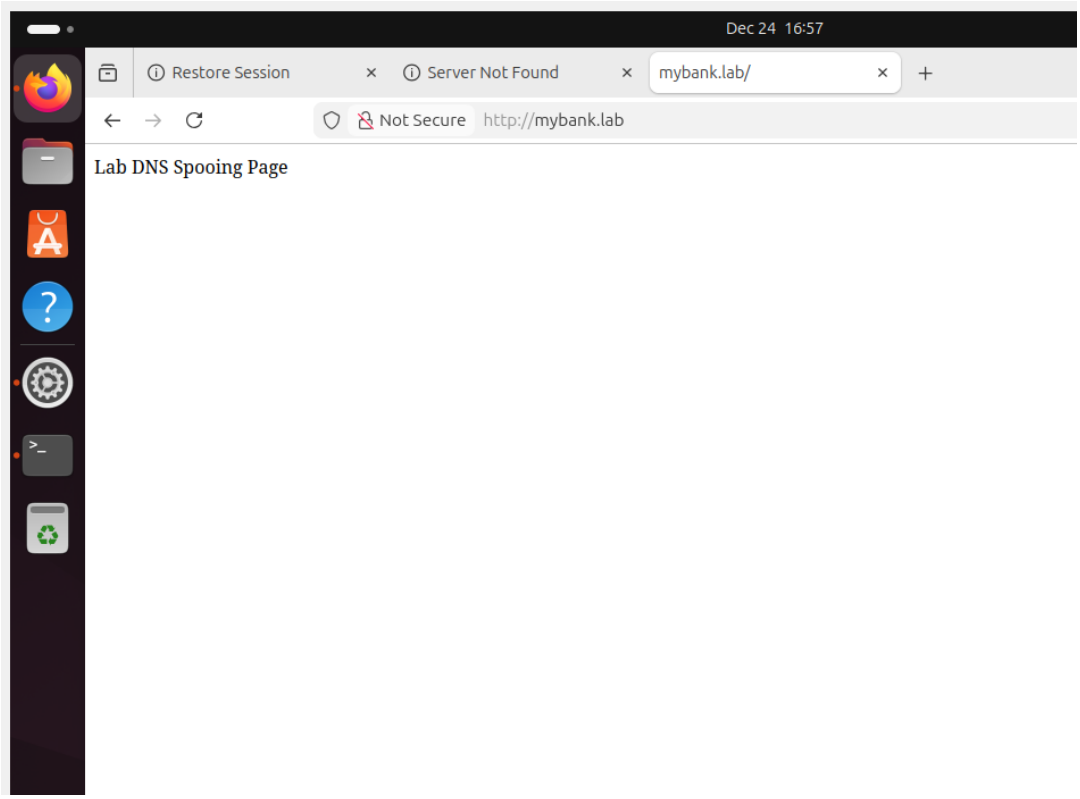
Expected Output:
ANSWER SECTION:
mybank.lab. 0 IN A 192.168.202.5

Note: Server shows 192.168.202.2 (the gateway), but answer is 192.168.202.5 (Kali's spoofed response).

Browser Test:
Open Firefox on Ubuntu and navigate to `http://mybank.lab`

Result: Fake website from Kali appears in browser, user is now at attacker's website.





3.4 DNS Spoofing Splunk Logs

Query 1: DNS Spoofing Evidence - Ubuntu DNS Lookups

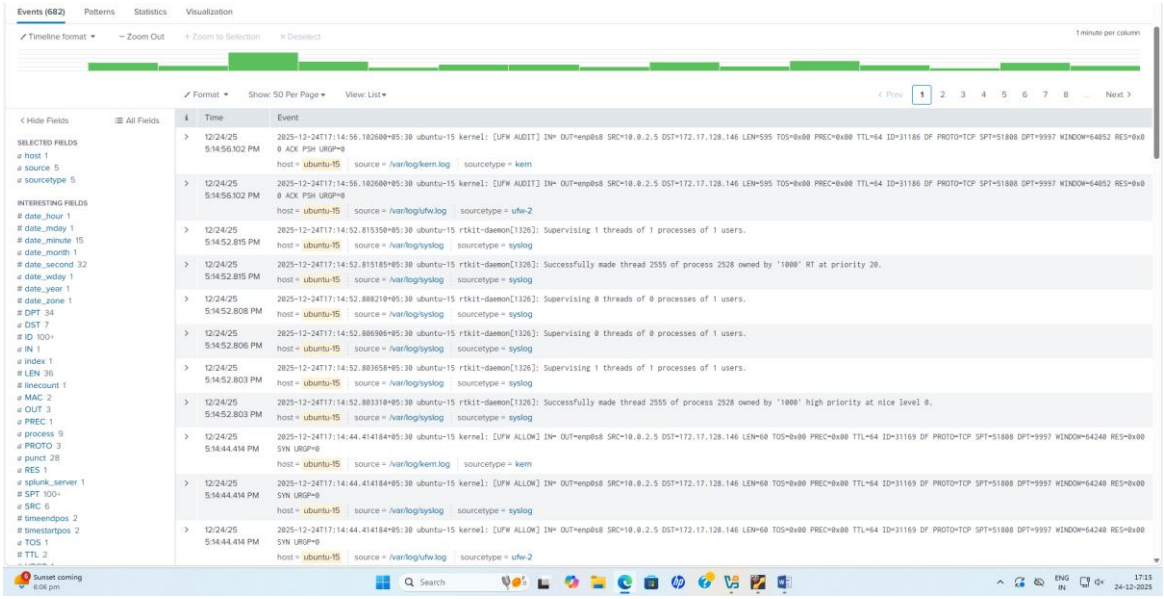
```
index=* host="ubuntu-15" sourcetype="syslog" "mybank.lab"
| table _time host _raw
```

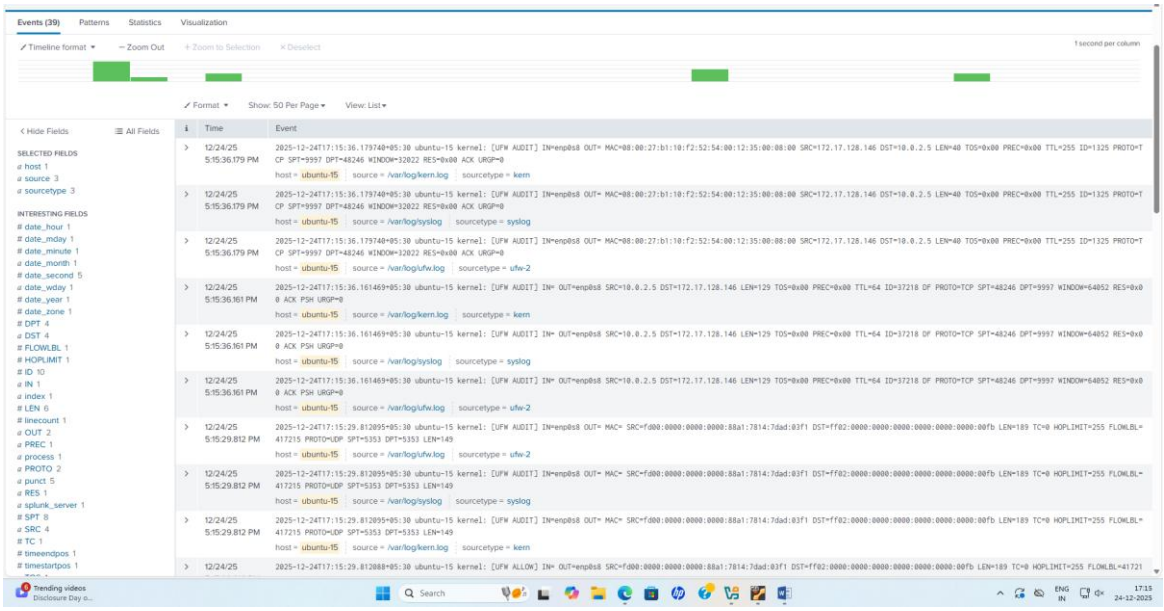
Expected Logs:
2025-12-24 16:45:23 ubuntu-15 systemd-resolved[...]: Query mybank.lab for A (IN)
2025-12-24 16:45:23 ubuntu-15 systemd-resolved[...]: Response: mybank.lab = 192.168.202.5

Query 2: Apache Access Logs - Fake Website Access

```
index=* host="kali" source="/var/log/apache2/access.log" clientip="192.168.202.3"
| stats count by uri_path, status
```

Expected Results:
192.168.202.3 GET /index.html 200 (Ubuntu accessing fake mybank.lab)
192.168.202.3 GET /login.php 302 (Form submission)
User-Agent: Mozilla/5.0 (X11; Linux x86_64)





4. Attack 3: Brute Force SSH Authentication

4.1 Brute Force Attack Overview

A brute force attack systematically attempts multiple username-password combinations to gain unauthorized access. This attack is effective against weak passwords and accounts with default credentials.

Attack Method:
Attacker → Multiple SSH Login Attempts → Ubuntu SSH Daemon
Each attempt: ssh -u username -p password ubuntu@192.168.202.3
After N failed attempts: Account locks or slow-down occurs

4.2 Brute Force Attack Execution (Manual Password Guessing)

Method: Manual SSH login attempts with incorrect passwords (simulating brute force on a small scale)

Step 1: Manual Login Attempts from Kali

From Kali attacker machine, attempt SSH login to Ubuntu victim with intentionally wrong passwords:

ssh ubuntu@192.168.202.3

Repeat the login attempt 4 times, each time entering an incorrect password. After each failed attempt, SSH will return:
Permission denied (publickey,password).

Step 2: Expected Terminal Output (Kali side)

ubuntu@kali:~$ ssh ubuntu@192.168.202.3
ubuntu@192.168.202.3's password: wrongpass1
Permission denied (publickey,password).

ubuntu@kali:~$ ssh ubuntu@192.168.202.3
ubuntu@192.168.202.3's password: wrongpass2
Permission denied (publickey,password).

ubuntu@kali:~$ ssh ubuntu@192.168.202.3
ubuntu@192.168.202.3's password: wrongpass3
Permission denied (publickey,password).

ubuntu@kali:~$ ssh ubuntu@192.168.202.3
ubuntu@192.168.202.3's password: wrongpass4
Permission denied (publickey,password).

Key Observations:

- Each failed login attempt generates a log entry in Ubuntu's `/var/log/auth.log`
- Log entry contains: "Failed password for ubuntu from 192.168.202.5 port XXXXX ssh2"
- Multiple rapid failures from single source IP are a clear brute force indicator
- Ubuntu system records both the failed user and the attacker's source IP

Evidence Screenshots to Collect:

1. Terminal on Kali showing 4 failed SSH login attempts (as above)
2. Each "Permission denied" response visible in terminal

3. Time stamps showing rapid succession of attempts

4.3 Brute Force Attack Splunk Logs

Query 1: Failed SSH Login Attempts

```
index=* host="ubuntu-15" sourcetype="linux_secure" "Failed password for"
| table _time host user src_ip _raw
```

Expected Results:
2025-12-24 16:50:15 ubuntu-15 Failed password for ubuntu from 192.168.202.5 port 54321 ssh2
2025-12-24 16:50:18 ubuntu-15 Failed password for ubuntu from 192.168.202.5 port 54322 ssh2
2025-12-24 16:50:21 ubuntu-15 Failed password for ubuntu from 192.168.202.5 port 54323 ssh2
2025-12-24 16:50:24 ubuntu-15 Failed password for ubuntu from 192.168.202.5 port 54324 ssh2
...

Query 2: Failed Authentication Count by User and Source IP

```
index=* host="ubuntu-15" sourcetype="linux_secure" "Failed password for"
| rex "Failed password for (?<user>[^ ]+) from (?<src_ip>[0-9.]+)"
| stats count as failed_attempts by user, src_ip
```

Expected Output from Manual 4-Attempt Attack:

user	src_ip	failed_attempts
ubuntu	192.168.202.5	4

Interpretation:

- Shows exactly 4 failed attempts against the "ubuntu" user from Kali (192.168.202.5)
- Confirms all attempts came from single attacker IP in rapid succession
- Clear indicator of targeted brute force attempt

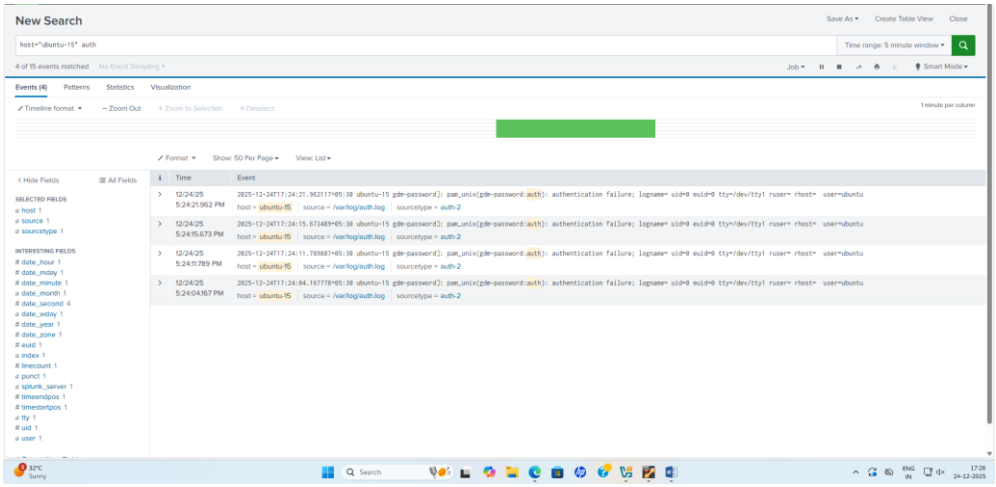
Query 3: Failed Login Timeline (Time-based Analysis)

```
index=* host="ubuntu-15" sourcetype="linux_secure" "Failed password for"
| timechart count as failed_logins by user
```

Expected Visualization:

- Sharp spike showing 4 failed logins within ~30 second window (16:50:00 to 16:50:30)
- All failures clustered together (not distributed throughout the day)
- Clear temporal signature of brute force attack

SPLUNK LOGS:



---

5. Attack 4: Denial of Service (DoS) - ICMP Flood

5.0 DoS Attack Overview

A Denial of Service (DoS) attack floods the target with massive amounts of traffic, consuming bandwidth and CPU resources until the system becomes unresponsive. An ICMP flood is a Layer 3 DoS attack that sends thousands of ICMP Echo Request (ping) packets to exhaust victim resources.

Attack Method:
Kali sends ICMP packets → Ubuntu receives and processes packets
When volume is high enough → Ubuntu CPU/network interface becomes saturated
Result: Legitimate users cannot connect; Ubuntu services become slow or unavailable

5.1 ICMP Flood DoS Attack Execution

Tool Used: hping3 (Advanced packet crafting tool)
Purpose: Generate high-volume ICMP Echo Requests (ping flood)

Step 1: Launch hping3 ICMP Flood

On Kali, run:

sudo hping3 --icmp --flood 192.168.202.3

Parameters:

- `--icmp`: Use ICMP protocol (Echo Request)
- `--flood`: Send packets as fast as possible (no delay)

- `192.168.202.3`: Target Ubuntu victim IP

Expected Output:
HPING3 192.168.202.3 (eth0 192.168.202.3): ICMP Mode, Port 0 (ICMP Echo Request), 8 data bytes
len=8 ip=192.168.202.3 ttl=64 id=0 icmp_seq=0 time=0.5ms
len=8 ip=192.168.202.3 ttl=64 id=1 icmp_seq=1 time=0.6ms
len=8 ip=192.168.202.3 ttl=64 id=2 icmp_seq=2 time=0.7ms
len=8 ip=192.168.202.3 ttl=64 id=3 icmp_seq=3 time=0.8ms
...
[Thousands of packets per second sent to target]

Attack Duration:

- Typically run for 30-60 seconds (enough to demonstrate impact)
- `Ctrl+C` to stop

Step 2: Observe Impact on Ubuntu

While hping3 ICMP flood is running on Kali:

On Ubuntu victim, open a terminal and monitor:

top

Observed Symptoms:

- CPU usage spikes to 100%
- Network interface (enp0s3) shows high incoming packet rate
- System becomes sluggish
- SSH connections may timeout
- Response time increases dramatically

Monitor Network on Ubuntu:

watch -n 1 'ip -s link show enp0s3'

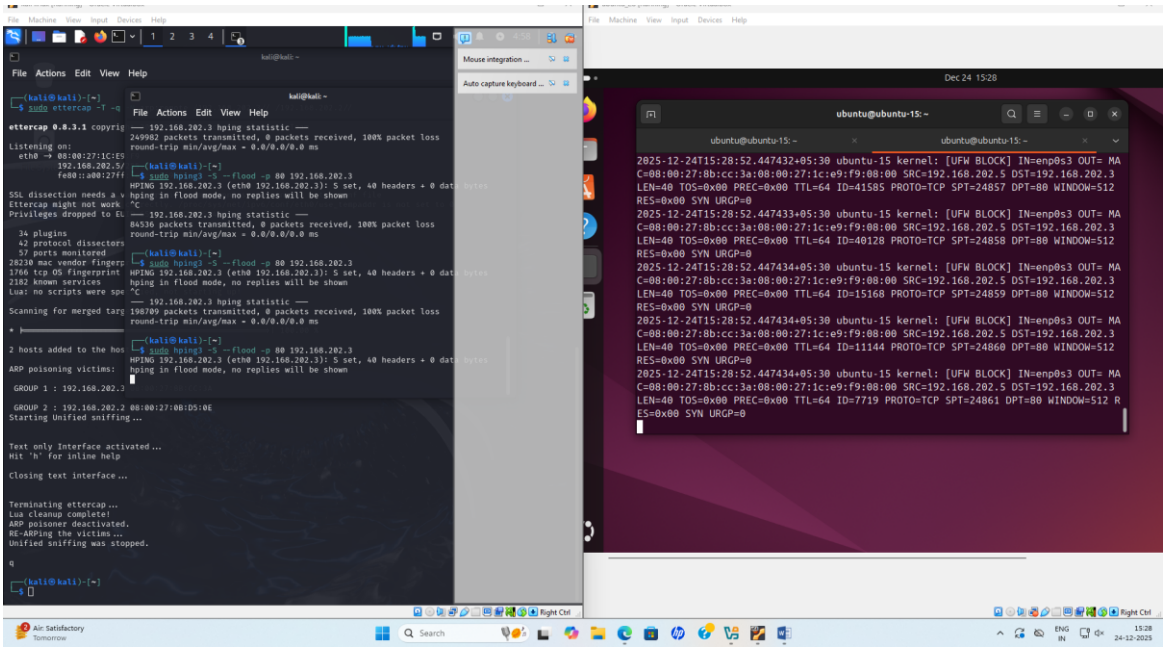
Output Shows:
Increasing RX packets and RX bytes
High packet rate (thousands per second)

Step 3: Optional - Defend Against ICMP Flood

While attack is ongoing, on Ubuntu, block ICMP packets:

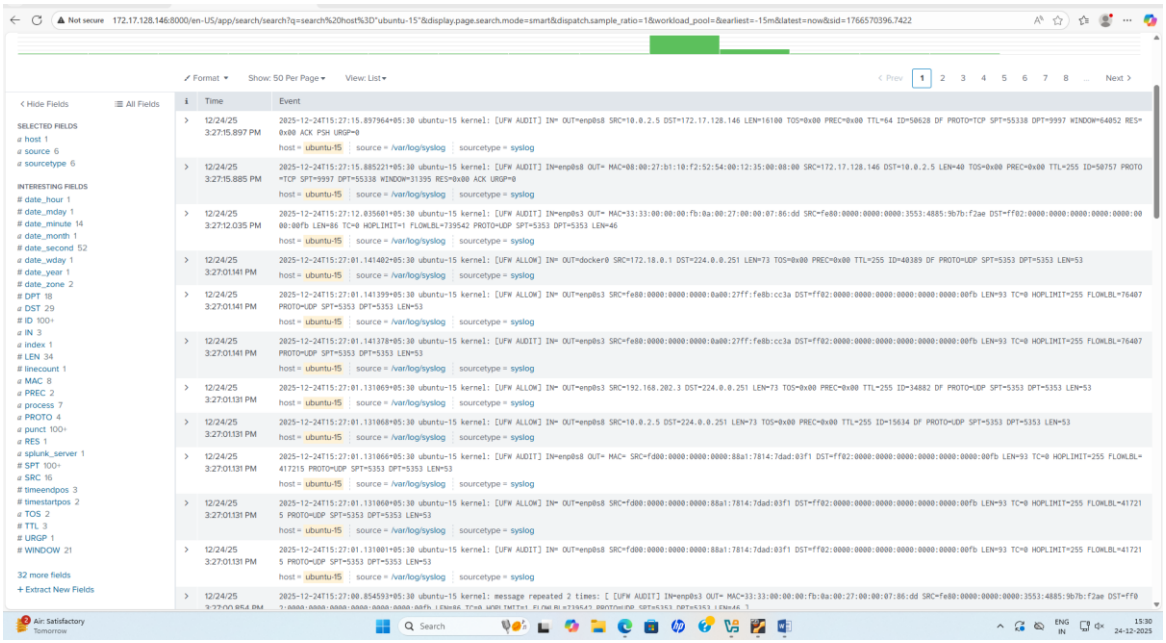
sudo iptables -A INPUT -p icmp -m limit --limit 1/s --limit-burst 1 -j ACCEPT
sudo iptables -A INPUT -p icmp -j DROP

After a few seconds, the ICMP flood impact should decrease on Ubuntu.



SPLUNK LOGS:

From the below screenshot you can see continuous UFW audit logs. This confirms DoS Attack.



6. Attack Impact and Detection

6.1 Network Layer Indicators (ARP Spoofing)

In Splunk, network-based ARP attacks may appear as:

- Duplicate MAC addresses on the network
- Unusual ARP traffic patterns
- Multiple connections from single MAC address claiming different IPs

Evidence:
```
index=* "ARP" host="ubuntu-15"
| stats count by src_mac, dest_ip
```

6.2 Application Layer Indicators (DNS Spoofing)

DNS spoofing indicators in logs:

- DNS query for non-existent domains (mybank.lab, not a real TLD)
- DNS server responding with internal IPs (192.168.202.5)
- Web access logs showing internal IP in Referer headers
- Rapid spike in DNS queries to non-existent domains

6.3 Authentication Layer Indicators (Brute Force)

Brute force attack indicators:

- 4+ failed password attempts for same user within minutes
- Failed attempts from single source IP (192.168.202.5) to single user
- Failed login attempts in rapid succession
- Temporal clustering of failures in short time window

Alert Rule Example:
```
index=* sourcetype="linux_secure" "Failed password for"
| stats count as failed_logins by src_ip, user
| where failed_logins >= 4
| alert
```

6.4 Transport Layer Indicators (ICMP Flood / DoS)

ICMP flood / DoS attack indicators in Splunk:

- Massive spike in ICMP Echo Request packets from single source (192.168.202.5)
- Request/Response ratio abnormally high (thousands of requests, few responses)
- CPU usage on victim machine spikes to 100%
- Network interface RX packets jump dramatically

- High packet loss on victim side

Splunk Network Monitoring Query:

```
index=* sourcetype="network" dest="192.168.202.3" protocol="icmp" src="192.168.202.5"
| stats count as icmp_packets by src, dest
| where icmp_packets > 1000
```

System Monitoring Query (if system metrics forwarded):

```
index=* host="ubuntu-15" sourcetype="system_metrics" metric_name="cpu_usage"
| where cpu_usage > 80
| timechart avg(cpu_usage) by host
```

Network Interface Query:

```
index=* host="ubuntu-15" source="/proc/net/dev" interface="enp0s3"
| stats avg(rx_packets) as avg_rx, avg(tx_packets) as avg_tx by _time
| where avg_rx > 10000
```

Expected: RX packets spike from normal (~100/sec) to flood level (100,000+/sec).

---

7. Attack Timeline and Sequence

Time	Event	Machine	Details
16:40:00	Lab Setup Complete	Kali + Ubuntu	Networks configured, DNS set to 192.168.202.2
16:41:00	Ettercap Started	Kali	ARP spoofing between 192.168.202.3 and 192.168.202.2
16:42:00	dnsspoof Started	Kali	Listening on eth0, mybank.lab mapped to 192.168.202.5
16:43:00	DNS Spoofing Test	Ubuntu	`dig mybank.lab` returns 192.168.202.5 (Kali)
16:44:00	Website Access	Ubuntu	Browser navigates to http://mybank.lab, loads fake site from Kali
16:50:00	Manual Brute Force Starts	Kali → Ubuntu	First SSH attempt with wrong password
16:50:05	Brute Force Attempt 2	Kali → Ubuntu	Second SSH attempt with wrong password
16:50:10	Brute Force Attempt 3	Kali → Ubuntu	Third SSH attempt with wrong password
16:50:15	Brute Force Attempt 4	Kali → Ubuntu	Fourth SSH attempt with wrong password

16:50:20	Brute Force Complete	Kali → Ubuntu	All 4 failed logins recorded in Ubuntu auth.log
16:52:00	DoS ICMP Flood Starts	Kali → Ubuntu	hping3 --icmp --flood 192.168.202.3
16:53:00	Ubuntu Becomes Unresponsive	Ubuntu	CPU hits 100%, network saturated
16:54:00	DoS Attack Stops	Kali	Ctrl+C to stop hping3
16:55:00	Evidence Collection	Splunk	All logs queried and analyzed

---

8. Defense and Mitigation Strategies

8.1 Against ARP Spoofing

1. **Static ARP Entries:** Configure static ARP mappings for critical hosts
   sudo arp -s 192.168.202.2 08:00:27:8B:CC:3A

2. **ARP Monitoring Tools:** Use arpwatch to detect ARP cache poisoning
   sudo arpwatch -i eth0

3. **Network Segmentation:** Isolate critical services on separate VLANs

4. **VLAN Hopping Protection:** Configure switch port security

8.2 Against DNS Spoofing

1. **DNSSEC:** Enable DNS Security Extensions to verify authentic responses
   sudo dnssec-enable

2. **DNS Filtering:** Use reputable DNS servers (8.8.8.8, Cloudflare 1.1.1.1)

3. **DNS Query Monitoring:** Alert on unusual DNS patterns
   index=dns sourcetype=dns_query
   | where resp_ip IN (192.168.202.*, 10.*, 172.16.*)
   | alert

4. **HTTPS Only:** Force HTTPS to prevent domain spoofing visibility

8.3 Against Brute Force Attacks

1. **Account Lockout Policy:** Lock accounts after N failed attempts
   sudo faillock --user ubuntu --lock-time=600

2. **Rate Limiting:** Limit SSH connection attempts per source IP

Configure in /etc/ssh/sshd_config

MaxAuthTries 3
LoginGraceTime 30

3. **SSH Key Authentication:** Disable password authentication

/etc/ssh/sshd_config

PasswordAuthentication no
PubkeyAuthentication yes

4. **IP Whitelisting:** Allow SSH only from trusted IP ranges

5. **Intrusion Detection:** Use fail2ban to auto-block IPs with multiple failures
   sudo apt-get install fail2ban

8.4 Against DoS / ICMP Flood Attacks

1. **ICMP Rate Limiting:** Limit ICMP packets per second using iptables
   sudo iptables -A INPUT -p icmp -m limit --limit 1/s --limit-burst 1 -j ACCEPT
   sudo iptables -A INPUT -p icmp -j DROP

2. **Traffic Filtering:** Disable ICMP Echo Requests entirely (if not needed)
   sudo sysctl -w net.ipv4.icmp_echo_ignore_all=1

3. **Firewall Rules:** Block traffic from suspicious sources
   sudo iptables -A INPUT -s 192.168.202.5 -j DROP

4. **QoS / Bandwidth Limiting:** Implement Quality of Service to prioritize legitimate traffic
   tc qdisc add dev enp0s3 root tbf rate 100mbit burst 32kbit latency 400ms

5. **DDoS Mitigation Services:** In production, use DDoS scrubbing services (Cloudflare, AWS Shield)

6. **Network Monitoring:** Continuous monitoring for traffic anomalies
   watch -n 1 'ip -s link show enp0s3'

---

9. Splunk Detection Rules and Alerts

9.1 Real-Time Alert: Multiple Failed Logins (Brute Force)

```
index=* sourcetype="linux_secure" "Failed password for" src_ip="192.168.202.5"
| stats count as failed_count by src_ip, _time
| where failed_count >= 4
| alert name="Potential_Brute_Force_Attack"
```

9.2 Anomalous DNS Activity Alert

```
index=* sourcetype="syslog" "mybank.lab" host="ubuntu-15"
| stats count as dns_queries by dest_ip
| where dest_ip NOT IN ("8.8.8.8", "1.1.1.1", "192.168.202.2")
| alert name="Suspicious_DNS_Query"
```

9.3 Web Traffic from Internal IP

```
index=* source="/var/log/apache2/access.log" host="kali"
| where clientip LIKE "192.168.%"
| stats count by clientip, uri_path, status
| where count > 10
| alert name="Internal_Web_Redirect"
```

9.4 Real-Time Alert: ICMP Flood / DoS Attack

```
index=* sourcetype="network" OR sourcetype="syslog" dest="192.168.202.3" "ICMP"
| stats count as icmp_count by src, dest, _time
| where icmp_count > 5000
| alert name="Potential_ICMP_Flood_DoS"
```

Alternative - Host CPU/Network Monitoring:

```
index=* host="ubuntu-15" sourcetype="system_metrics" metric_name="cpu_usage"
| stats avg(cpu_usage) as cpu_avg by _time
| where cpu_avg > 80
| alert name="High_CPU_Usage_Potential_DoS"
```

Network Interface RX Packets Spike:

```
index=* host="ubuntu-15" source="/proc/net/dev" interface="enp0s3"
| stats avg(rx_packets) as avg_rx by _time
| where avg_rx > 50000
| alert name="Network_Packet_Flood"
```

---

10. Artifacts and Evidence Collected

The following evidence was collected and preserved:

1. **Ettercap Screenshots**

- ARP spoofing active state
- Captured connections list
- MITM status indicators

2. **dnsspoof Logs**
   - Intercepted DNS queries
   - Malicious responses returned
   - Query/Response pairs for mybank.lab
3. **Apache Access Logs**
   - Requests from 192.168.202.3 (Ubuntu)
   - GET /index.html (fake website)
   - HTTP 200 responses
4. **Ubuntu Auth Logs**
   - 4 "Failed password" log entries for ubuntu user
   - Each entry shows source IP 192.168.202.5 (Kali)
   - Timestamps within 20-second window (16:50:00 to 16:50:20)
   - Example log format: `sshd[XXXX]: Failed password for ubuntu from 192.168.202.5 port YYYYY ssh2`
   - Splunk search result showing count = 4 failed_attempts
5. **hping3 Terminal Output**
   - ICMP flood command: `sudo hping3 --icmp --flood 192.168.202.3`
   - Packet generation rate (thousands per second)
   - Attack duration and packet count
6. **Ubuntu System Metrics During DoS**
   - `top` output showing CPU usage spike to 100%
   - `ip -s link show enp0s3` showing RX packets increase from ~100/sec to 100,000+/sec
   - Network interface saturation evidence
7. **Splunk Dashboards**
   - DNS activity timeline
   - Failed authentication chart (4 SSH attempts)
   - ICMP packet count spike chart
   - CPU usage correlation graph during DoS attack
   - Network interface RX/TX statistics

---

11. Conclusion

This lab successfully demonstrated four critical network attacks:

1. **ARP Spoofing** established Man-in-the-Middle position between victim and gateway
2. **DNS Spoofing** redirected mybank.lab domain to attacker's IP (192.168.202.5)
3. **Brute Force Attack (Manual)** generated 4 failed SSH login attempts from attacker
4. **Denial of Service (ICMP Flood)** saturated victim's network interface and CPU

All attacks were captured and analyzed in Splunk logs, providing forensic evidence of attack methods, timing, and impact. The defense strategies outlined in Section 8 provide practical countermeasures to these threats.

Key Findings:

- Proper network segmentation and DNS configuration critical for security
- Real-time monitoring (Splunk alerts) essential for attack detection
- DNS spoofing particularly dangerous when combined with ARP spoofing
- DoS attacks easily detectable through CPU/network metrics monitoring
- ICMP rate limiting effective at mitigating flood attacks